

Argus 설계 문서

반도체 팹 협력사 부품 교체 요청·승인 AI-Ready 워크플로우

설비 엔지니어가 부품 교체를 요청하면 AI 에이전트 3종(규격·법령·안전서류)이 검토 초안을 만들고, 엔지니어가 수정해 제출하면 안전관리자가 승인·거절하는 워크플로우. AI는 제안까지, 결정은 사람이 한다.

문서 버전	v1.0 (2026-09-04)
팀	광주 캠퍼스 2반 · 은태현팀 (5인)
PM · DA	은태현
Product/UX · FE	문승은
API Architect · BE	정구현
BE · AI-Ready · 발표	장병현
DevOps & Integration	신서현
저장소	github.com/Taeyum/skala-fixguide (코드 식별자: fixguide)
발표	2026-09-04 15:00 · 15분 + Q&A 5분

본 문서의 샘플 데이터·법령 요약은 설계 검증용 예시입니다. 저장소명·패키지명·시드 계정 등 코드 식별자는 초기 작업명 "fixguide"를 사용합니다.

목차

1	문서 개요	10	시스템 아키텍처
2	프로젝트 개요와 문제 정의	11	데이터 모델 (ERD)
3	범위	12	API 설계
4	팀 구성과 R&R	13	데이터 흐름과 핵심 트랜잭션
5	일정과 WBS	14	구현·검증 현황
6	요구사항	15	결정 기록
7	Use-Case	16	리스크·한계·로드맵
8	화면 정의	17	평가 루브릭 대응표
9	AI-Ready 설계	부록	실행 방법 · 시드 데이터 · 문서 지도

1. 문서 개요

1.1 목적

이 문서는 3일 미니프로젝트 "Argus"의 기획·설계·구현·검증을 한 권으로 묶은 최종 설계 문서다. 심사 기준(정량 Rubric 60% + Peer Review 40%)이 요구하는 산출물 — Use-Case, 와이어프레임, AI 확장 지점·프롬프트·JSON 스키마, 시스템 다이어그램, ERD, REST API 명세, Mock API, FE/BE/DB 연동 — 을 각 장에 배치했고, 마지막 17장에 루브릭 항목별 대응 위치를 표로 정리했다.

1.2 읽는 법

- 심사자: 2장(문제) → 7장(Use-Case) → 9장(AI-Ready) → 10·11·12장(설계) → 14장(검증) → 17장(루브릭 대응) 순으로 읽으면 채점 항목을 따라간다.
- 개발자: 11·12·13장이 구현 기준이다. 세부 명세는 저장소 docs/ 의 원본 문서(부록 C)를 본다.

1.3 문서 이력

버전	일자	변경
v0.1	09-02	아이디어 후보 조사, 페르소나·문제 정의, AI 기능 정의, 범위·R&R 확정
v0.5	09-03	화면정의서 v2.0, API 명세서 v1.0, ERD v3, docker-compose. 오후 개발 착수
v1.0	09-04	구현·검증 결과 반영, AI-Ready 검토, ERD-코드 정합성 검증, 루브릭 대응표

2. 프로젝트 개요와 문제 정의

2.1 한 줄 정의

Argus는 반도체 팹 협력사의 설비 엔지니어가 부품 교체를 요청하면, AI 에이전트가 규격·법령·안전서류 검토 초안을 만들고, 엔지니어가 확인·수정해 제출하면 안전관리자가 승인·거절하는 B2B 워크플로우 시스템.

2.2 배경

반도체 공정(증착·식각 등)에서는 특수가스를 취급하며, 유량 이상 등 알람이 발생하면 배관·밸브·필터 같은 부품을 교체해야 한다. 교체 자체는 짧지만, 그 전에 **대체 부품의 규격 적합성 확인 → 적용 법령·사내 절차 검토 → 작업허가서·위험성평가 작성 → 안전관리자 승인**이 필요하다. 펌은 유해위험물질 다량 취급 사업장이라 공정안전관리(PSM)의 변경관리 절차와 고압가스안전 관리법 등이 걸리고, 이 절차는 법으로 요구되는 것이라 생략할 수 없다.

대기업(SK하이닉스)은 자율형 펌 2030, GaiA 장비보전 에이전트, AMOS 대화형 장애 분석 등으로 "알람 → 분석 → 조치"를 이미 자동화하고 있다. 반면 2·3차 협력사(패키징·소부장)는 GPU·데이터 전담 조직이 없고 외부 클라우드 LLM을 쓸 수 없어 이 흐름이 사람 손에 남아 있다. SK하이닉스가 2026-07 협력사 스마트팩토리 전환 컨설팅을 2·3차까지 확대(5년 1.4조)한 것이 이 시장의 근거다.

2.3 왜 "알람 분석"이 아니라 "변경 승인"을 잡았나

초기 후보는 "알람 → 원인·정비 가이드"였다. 그러나 조사 결과 그 영역은 이미 상용화돼 있었다. 산업용 스마트글래스 정비 가이드(RealWear·Vuzix·버넥트), 대화형 장애 분석(하이닉스 AMOS), 그리고 범용 코딩 에이전트에 로그를 붙여넣어도 원인 후보가 나온다. 반면 "조치가 결정된 뒤 승인까지 걸리는 시간"은 확인된 상용 솔루션이 없었고, 팀원의 현업 경험에서 가장 반복적으로 언급된 병목이었다.

As-Is 단계	소요	병목 원인	AI로 줄일 수 있나
① 알람 확인·교체 필요 판단	수 분	—	이미 상용화 영역
② 대체 부품 규격·호환 확인	반나절	영문 매뉴얼, 벤더 문의	가능 (A1)
③ 적용 법령·사내 절차 확인	1일	조문 검색, 선배 문의	가능 (A2)
④ 작업허가서·위험성평가 작성	1일	빈 양식부터, 누락 반복	가능 (A3)
⑤ 안전관리자 검토·반려 왕복	2~3일	누락·오적용 발견 → 반려	준비 품질 향상으로 왕복 감소
⑥ 승인 결정	수 시간	법적 책임	불가 — 사람이 결정
⑦ 외부기관 검사 (해당 시)	수 일~주	가스안전공사 등	불가 — 대상 아님

이 시스템은 ②③④⑤를 대상으로 한다. ⑥ 승인 자체와 ⑦ 외부기관 리드타임은 줄이지 못하며, 이를 발표에서 명시한다. "승인 단축"이 아니라 "승인 준비 시간 단축 + 반려 왕복 감소"가 정확한 목표다.

2.4 페르소나

	요청자 — 김민준 (29, 2차 협력사 공정 설비 엔지니어 2년 차)	승인자 — 박안전 (환경안전팀 안전관리자)
상황	야간 라인 단독 근무 중 알람. 교체 부품을 정했지만 "이거 변경관리 대상인지, 어떤 법령이 걸리는지, 서류는 뭘 써야 하는지" 모름	하루 여러 건의 변경 요청 검토. 서류 누락·법령 오적용을 잡느라 시간 소진. 반려하면 왕복, 그냥 승인하면 본인 책임
지금 하는 방법	수백 쪽 영문 매뉴얼 검색 20~40분, 수면 중인 선배에게 전화, 엑셀에 수기 기록(누락 잦음)	같은 실수가 반복된 서류를 또 검토. 반려 사유를 메신저로 따로 전달
가장 아픈 순간	동일 사양 교체인지 변경인지 애매해서 일단 다 올리고 반려당함	"이 판단의 근거가 뭐냐"를 물어볼 데가 없음
원하는 것	원인 후보와 근거, 뭘 올려야 하는지 처음부터	누락이 걸려진 서류, 근거가 붙어 있는 판단, 거절 사유가 요청자에게 그대로 전달
KPI	원인·근거 파악 30분 → 5분	승인 준비 시간, 반려 횟수, 거절 사유 유형별 분포

구매 논리의 중심은 안전관리자다. 안전관리자가 검토에 쓰는 시간과 반려 횟수가 곧 사업 KPI이며, 구매자는 협력사 생산·환경 안전(EHS) 조직이다.

2.5 검증 가설

협력사에서 부품 교체는 규격·법령·안전서류 검토를 엔지니어가 처음부터 하고, 안전관리자가 누락을 잡아 반려하는 왕복이 반복된다. 요청 컨텍스트(설비·물질·운전조건·제품명·스펙)를 AI 에이전트 3종이 검토 초안으로 만들고 엔지니어가 수정해 제출하면, 서류 준비 시간과 반려 횟수를 줄여 승인까지의 기간을 단축할 수 있다.

PoC에서 증명할 것은 AI 정확도가 아니라 **설계**다. (a) AI가 들어올 자리가 서비스 흐름상 타당한가, (b) 입출력 JSON 스키마가 기존 웹 구조와 호환되는가, (c) Mock으로 화면·API·DB가 End-to-End로 도는가.

2.6 설계 원칙

원칙	적용
AI는 제안, 결정은 사람	AI는 초안·분류·누락 표시까지. 제출은 엔지니어, 승인은 안전관리자. 금융위 AI 가이드라인의 '보조수단성'과 같은 구조
사실 / 추론 / 행동 분리	입력(work_requests) · AI 산출(agent_runs, agent_steps, agent_results) · 사람의 결정(approvals)을 테이블로 분리. 위 층이 아래 층을 덮어쓰지 않음
AI 원본 보존	AI 원본(original_json)과 엔지니어 수정본(payload_json)을 따로 저장. "AI가 뭐라 했고 사람이 뭘 바꿨나"가 남음
온프레미스 전제	협력사는 외부 클라우드 LLM 불가. ai_configs.egress_allowed 기본 false. Mock → 사내 LLM 교체 지점을 클래스 하나로 고정
Interface First	FE는 JSON 계약만 안다. MockAgentEngine을 실제 엔진으로 바꿔도 FE·API·DB 계약은 불변

3. 범위

3.1 이번 PoC (3일)

영역	포함	제외 (Phase 2)
사용자·인증	회원가입, 로그인(JWT), 역할 2종(ENGINEER / SAFETY_MANAGER), 로그아웃(토큰 블랙리스트)	SSO, 조직·테넌트 전환 UI
요청	등록(임시저장·정식), 제품 유형 5종 동적 스펙, 제품 사진 업로드(최대 5장), 수정, 내 요청 목록	설비·부품 마스터 연동, 유사 이력 추천
AI 검증	A1 규격·호환 / A2 법령·조문 / A3 안전서류 3종. 비동기 실행(202) + 폴링. 전부 Mock	실제 LLM, 법령 RAG, A1 호환표 연동, A4 벤더 견적
결과 수정	A1·A2 항목 추가·수정·삭제, A3 문서 본문 편집, 엔지니어 설명, 제출(검증 422)	항목 단위 API, 동시 편집
승인	안전관리자 큐, 상세 검토(읽기 전용), 승인 / 거절(사유 필수), 재제출	체크리스트 blocking(v2.0에서 제거), 다단계 결재
대시보드	역할별 KPI, 거절 사유 TOP5	평균 승인 소요시간(v2.0에서 제거), 에이전트 채택률
인프라	Docker Compose (PostgreSQL 16, Redis 7, BE, FE), 시드 데이터, Swagger, Postman Mock	Kubernetes 배포, Flyway, Testcontainers

3.2 화면·API·테이블 규모

화면	Use-Case	API	테이블	AI 확장 지점	테스트
9 (공통 2 · 엔지니어 5 · 안전관리자 2)	13	16	8	3 (+채택률 축적 1)	통합 38 + 스키마 정합성 1

4. 팀 구성과 R&R

가이드의 7개 역할을 5명이 나눠 맡았다. 한 사람이 두 역할을 겸하되, 각 역할의 산출물이 저장소 경로와 커밋으로 남도록 했다.

가이드 역할	담당	책임	주요 산출물 (저장소 경로)
PM · Data Architect	은태현	일정·진행 추적, 발표 총괄, PR 병합·구조 통합, ERD·정규화, DDL	docs/01_planning/ , docs/06_erd/ , 병합 커밋 3건, 이 문서
Product/UX Designer · Frontend	문승은	페르소나·Use-Case·User Flow, 화면정의서, 와이어프레임, Vue 화면	docs/02_usecase/ , docs/03_wireframe/ , 화면 정의서 v2.0, frontend/
API Architect · Backend	정구현	REST API 명세, 에러 코드, Postman Mock, 요청·승인 도메인 BE	docs/07_api/README.md , Postman 컬렉션·Mock, backend/.../workrequest , approval
Backend · AI-Ready · 발표	장병현	SK AX 사례 조사, 인증·대시보드 BE, AI 확장 지점·프롬프트·JSON 스키마, 발표	docs/05_ai_ready/ , backend/.../auth , agent , 발표
DevOps & Integration	신서현	GitHub 레포·브랜치 규칙, Docker Compose, E2E 연동 검증	docker-compose.yml , infra/ , .github/ , E2E 검증 보고서, docs/09_qa/

4.1 협업 규칙

- **브랜치**: main 보호, 기능은 feat/{도메인}-{요약} , 문서는 docs/{번호}-{요약} . PR 필수, 리뷰 1명.
- **커밋**: feat / fix / docs / chore / test prefix. 각자 이름으로 커밋이 남도록 (R&R 증빙).
- **계약 우선**: 상태값·필드명·에러 코드는 API 명세서 2장·8장이 단일 진실. FE 타입, BE enum, Swagger, Postman이 전부 이를 따른다.
- **체크포인트**: Day 2 오후 FE-BE 연동 확인, Day 3 12:00 코드 프리즈, 리허설 2회.

4.2 실제 운영에서 조정된 것

Day 2 오후에 백엔드 PR 두 건(인증·대시보드 / 요청·AI·승인)이 서로 다른 뼈대(PM의 초기 Spring Boot 4.1 스캐폴드 vs 팀원의 Spring Boot 3.3.5 com.skala.fixguide 구조)에서 작성된 것이 드러났다. PM이 팀원 구현체를 채택하고 다른 PR을 그 구조로 포팅해 병합했다(15장 결정 기록 D-07). 그 결과 백엔드가 계획보다 하루 먼저 완성돼 Day 3 오전 인력을 프론트엔드와 문서 정리에 재배치했다.

5. 일정과 WBS

가이드의 3일 로드맵(Day 1 기획·아키텍처 → Day 2 설계·스캐폴딩 → Day 3 검증·발표)을 기준으로 잡되, 실제로는 Day 2 오전에 설계 문서(API·ERD·Compose·Use-Case·Wireframe)를 마감하고 오후부터 개발에 들어갔다. 아래 WBS는 **계획 vs 실제**를 함께 적었다.

5.1 Epic 0 — 사전 준비 (09-01 ~ 09-02 오전)

ID	업무	담당	선행	산출물	결과
PRE-1	팀 편성, 역할 배정, 커뮤니케이션 채널	PM	—	R&R 초안	완료
PRE-2	가이드·루브릭 정독, 채점 항목 → 산출물 매핑	전원	—	루브릭 체크리스트	완료 (17장)
PRE-3	GitHub 레포 생성, 폴더 틀(docs/01~09 , backend, frontend, infra), 브랜치 규칙	DevOps	PRE-1	빈 레포 + README	완료
PRE-4	SK AX 2025-26 사업·고객 사례 조사 (금융 12건, 제조·반도체 9건, 하이닉스 7건)	장병헌	—	배경 조사 문서, 후보 A~O	완료
PRE-5	업계 Pain Point 조사 (SK AX 임원 발언, 금융 규제, PoC→운영 실패 데이터, 제조 암묵지)	PM	—	Pain Point 조사 자료	완료

5.2 Epic 1 — Day 1 (09-02) 서비스 기획 & Architecture 정의

ID	업무	담당	선행	산출물	결과
D1-1	브레인스토밍 → 후보 압축 (E 설비 정비 가이드 / N 불완전판매 검수)	전원	PRE-4	후보 비교표	E 채택
D1-2	교수 상담 (기획 프레임, Mock 범위, 비동기 수준, 멀티테넌트, R&R) — 질문 15개	장병헌	D1-1	상담 기록	Mock 기준 채점·큐 불필요 확인
D1-3	문제 재정의: 알람 분석 → 조치 이후 변경 승인 병목 . 못 줄이는 것(승인 자체·외부 검사) 명시	PM	D1-1	기획서 2장	완료
D1-4	페르소나 2명(설비 엔지니어·안전관리자) + 검증 가설	UX	D1-3	페르소나 시트	완료
D1-5	AI 기능 정의: 확장 지점 3곳(A1 규격·호환, A2 법령, A3 안전서류), A4 벤더·호환표 연동은 Phase 2	장병헌·정구현	D1-3	AI 확장 지점 정의서	완료
D1-6	개발 범위 결정: 회원가입 X→O(후 변경), 로그인, 메인 2종, 엔지니어 4화면, 안전관리자 2화면	PM	D1-5	범위 표 (3장)	완료
D1-7	Actor 3종 · Use-Case 13개 정의, include/extend, AI 확장 지점 표시	UX·정구현	D1-5	docs/02_usecase/usecase_spec.md , FigJam 다이어그램	완료
D1-8	화면정의서 v1.1 → v2.0 (평균 승인시간·진행률 제거, 결과 수정 가능, 제품 사진, 유형별 스펙, 체크리스트 blocking 제거)	UX	D1-7	WRA 화면정의서 v2.0 (9화면)	완료

ID	업무	담당	선행	산출물	결과
D1-9	아키텍처 방향: 모듈러 모놀리스 + 바운디드 컨텍스트, MSA 미채택 근거, agentrun 분리 1순위	PM	D1-5	아키텍처 다이어그램 v1	완료
D1-10	스택 결정: Spring Boot 3.3 + Vue 3 + PostgreSQL 16 + Redis, JWT	PM·BE	D1-9	기술 스택 표	완료 (D-07 참조)
D1-11	Day 1 스탠드업 — 산출물 점검, Day 2 오전 문서 마감 합의	PM	전부	Day 1 체크리스트	완료

5.3 Epic 2 — Day 2 (09-03) 시스템 설계 & Scaffolding

ID	업무	담당	선행	산출물	결과
오전 — 설계 문서 마감					
D2-1	REST API 명세서 v1.0: 공통 규약, enum, 상태 전이, 16 API 상세, 에러 코드, DB 매핑, 정합성 메모 10건	정구현	D1-8	docs/07_api/README.md	완료
D2-2	ERD v3: 대리키 PK + 자연키 UNIQUE, 8테이블, jsonb·enum, 정규화 근거. DBML + 정의서	PM(DA)	D2-1	docs/06_erd/	완료
D2-3	ERD ↔ 명세 정합: requestNo 신설, AI_DONE 상태, 제출 경로 확정, 전체 치환 방식	PM·정구현	D2-2	명세 9장 정합성 메모	완료
D2-4	Docker Compose: PostgreSQL 16 · Redis 7 · BE · FE, .env.example, 프로파일 분리 (local/H2, docker/PG, test)	DevOps	D1-10	docker-compose.yml, infra/	완료
D2-5	와이어프레임 (Figma) 9화면 + User Flow, UI 목업	UX	D1-8	docs/03_wireframe/	완료
D2-6	AI 프롬프트 3종 초안 + 입출력 JSON 스키마 4종 + Playground 검증	장병현	D1-5, D2-1	docs/05_ai_ready/	완료
오후 — 개발 착수					
D2-7	BE 스캐폴딩 + 인증·대시보드·요청 목록 (PR #1), 회원가입·로그아웃 (PR #3)	장병현	D2-1, D2-4	통합 테스트 21→34	병합 c049e75, d0e66f1
D2-8	요청 등록·상세·수정·제출, 사진, AI 실행·폴링·결과 수정, 승인 (PR #4/#5)	정구현	D2-1	명세 5, 7~15	포팅 병합 5ada3d2
D2-9	PR 구조 통합: 초기 뼈대 폐기, com.skala.fixguide 구조로 포팅, requestNo 채번·상태 전이 메서드, 권한 정책 클래스	PM	D2-7, D2-8	테스트 38 통과	완료
D2-10	시드 데이터: 계정 3, 요청 7(상태별), AI 실행 6, 사진 2, 승인 5 → 16 API 즉시 테스트 가능	PM	D2-9	SeedDataInitializer	956568d
D2-11	FE 스캐폴딩: Vue 3 + Vite + Router + Pinia + axios, 라우트 맵 9개, 레이아웃 2종	문승은	D2-5	FE 개발 계획서, 골격	완료
D2-12	E2E 검증 1차: 생성 → 202 → 폴링 → 검토 → 제출(422→200) → 승인 → APPROVED, curl 로그	DevOps	D2-9	E2E 검증 보고서 (문서 11)	완료

ID	업무	담당	선행	산출물	결과
D2-13	Day 2 스탠드업 — FE-BE 연동 체크포인트, 스타크 변경(FastAPI→Spring) 문서 반영 목록	PM	전부	Day 2 체크리스트	완료

5.4 Epic 3 — Day 3 (09-04) 설계 검증 & 최종 발표

ID	업무	담당	선행	산출물	결과
오전 — 산출물 정리·검증					
D3-1	Swagger Pageable 파라미터 분리, 잘못된 sort 400 처리	정구현	D2-9	62a9def	완료
D3-2	Postman: OpenAPI 임포트 → 컬렉션 → 공개 Mock 서버 배포, README에 URL·curl	PM	D3-1	Mock URL	완료
D3-3	AI-Ready 산출물 검토 (5축 A~E, 발견 13건, 심각도 분류) → HIGH 반영	장병헌	D2-6	docs/09_qa/ai_ready_review.md	완료
D3-4	ERD ↔ 코드 정합성: Hibernate DDL 추출, DBML·정의서 컬럼 대조(8테이블 전부 일치), SchemaDocConsistencyTest	PM	D2-2, D2-9	docs/06_erd/schema_postgres.sql, 테스트	완료
D3-5	명세 7장 DB 매핑 갱신 (ai_configs, input_snapshot, original_json, request_no)	정구현	D3-4	명세 v1.0 갱신	완료
D3-6	FE 핵심 화면: 로그인 → 목록 → 상세(폴링·결과 수정) → 승인 큐·상세	문승은·전원	D2-11	Vue 화면	진행
D3-7	데모 시나리오 대본(90초), 시드 상태별 진입 경로, 노트북 2대 환경 복제, 백업 영상	DevOps·PM	D3-6	데모 스크립트	진행
D3-8	발표 슬라이드 (가이드 6섹션: 기획 3분 · AI-Ready 2분 · 설계 4분 · 데모 4분 · 회고 2분)	PM·장병헌	D3-3~5	슬라이드	진행
D3-9	이 설계 문서 최종본 (PDF)	PM	전부	본 문서	완료
D3-10	Peer Review 질의 1개, 4개 영역 채점표 양식	전원	—	질의안	진행
오후					
D3-11	12:00 코드 프리즈 → 13:00 리허설 2회차 → 14:30 장비 세팅	전원	D3-6~8	—	예정
D3-12	15:00 Project Pitch & Live Demo (15분 + Q&A 5분)	장병헌(발표)	전부	발표	예정

5.5 계획 대비 실제

항목	계획	실제	이유·영향
백엔드 완성	Day 3 오전	Day 2 저녁 (16 API, 38 테스트)	팀원 두 명이 병렬로 구현. 구조 충돌은 PM이 포팅으로 해결
프론트엔드	Day 2 오후 핵심 화면 2개	Day 3 오전 집중	백엔드 조기 완성으로 인력 재배치. 데모 리스크는 시드 데이터로 완화

항목	계획	실제	이유·영향
스택	FastAPI 검토	Spring Boot 3.3.5 채택	실제 구현이 먼저 나온 쪽을 채택 (D-07)
Postman Mock	Day 2	Day 3 오전	백엔드가 먼저 완성돼 Mock 서버 필요성이 낮아졌으나 루브릭 항목이라 추가
회원가입	제외	포함	역할 선택·데모 설득력을 위해 추가. 비용 2시간

6. 요구사항

6.1 기능 요구사항

ID	요구사항	화면	API	테이블
REQ-F-001	이메일·비밀번호·역할로 회원가입. 이메일 중복 409, 비밀번호 확인 불일치 400	C_01	1	users
REQ-F-002	로그인 후 역할에 따라 엔지니어 홈 / 안전관리자 큐로 분기. 토큰 만료·위조 401	C_00	2, 3, 16	users
REQ-F-003	엔지니어는 본인 요청만, 안전관리자는 PENDING 이상 요청 전체 조회. 위반 403	전 화면	6, 7	work_requests
REQ-F-004	설비·라인·물질·운전조건·제품명·제품 유형·유형별 스펙·증상·메모·제품 사진(≤5장, ≤10MB)으로 요청 등록. 임시 저장(DRAFT) 지원	E_02	5, 8, 9	work_requests, work_request_photos
REQ-F-005	제품 유형 5종(밸브·피팅·튜브·레귤레이터·필터·기타)별 필수 스펙 키를 서버가 검증. 불일치 400	E_02	5	work_requests.spec_json
REQ-F-006	AI 검증 3종(A1 규격·호환, A2 법령·조문, A3 안전서류)을 비동기 실행. 202 + 폴링, 진행 중 중복 실행 409	E_02→E_03	11, 12	agent_runs, agent_steps, agent_results
REQ-F-007	AI 결과를 엔지니어가 수정(A1·A2 항목 추가·수정·삭제, A3 본문 편집). AI 원본은 보존	E_04	13	agent_results (payload_json / original_json)
REQ-F-008	엔지니어 설명 필수, 3종 결과 존재, A2 법령 1건 이상일 때만 제출 가능. 미충족 422 + fieldErrors	E_04	14	work_requests
REQ-F-009	안전관리자는 요청 정보·사진·AI 결과(읽기 전용, 수정 표시)·엔지니어 설명을 보고 승인 또는 거절(사유 10자 이상 필수)	S_01, S_02	15	approvals
REQ-F-010	거절 사유가 요청자에게 그대로 전달. 수정 후 재제출 시 이전 결정 이력 보존	E_05, E_04	7, 14	approvals (append-only)
REQ-F-011	역할별 KPI(엔지니어: 작성중·AI검증중·승인대기·반려 / 안전관리자: 승인대기·오늘처리·이번달 승인·거절)와 거절 사유 TOP5	E_01, S_01	4	work_requests, approvals
REQ-F-012	모든 요청은 업무 번호 WR-YYYYMMDD-NNN 을 가지며 화면에 표시된다	전 화면	5, 6, 7	work_requests.request_no

6.2 비기능 요구사항

ID	요구사항	구현
REQ-NF-001	AI 응답 지연이 UI를 막지 않는다 (Asynchronous Pipeline)	202 Accepted + pollIntervalMs 2500 + allDone
REQ-NF-002	AI 구현체를 교체해도 FE·API·DB 계약이 바뀌지 않는다 (Interface First)	MockAgentEngine 단일 교체 지점, AgentService·DTO 불변
REQ-NF-003	AI 결과가 DB에 그대로 저장되고 FE로 전달된다 (Structured Data)	payload_json 스키마 = API 응답 agentRun.results[] 구조
REQ-NF-004	프롬프트·모델·키가 코드 밖에 있다 (Security & Config Isolation)	ai_configs 테이블 + .env, egress_allowed 기본 false
REQ-NF-005	모든 4xx·5xx는 단일 에러 포맷	ErrorResponse (timestamp, status, code, message, path, fieldErrors)
REQ-NF-006	한 명령으로 전체 스택 기동	docker compose up -d --build → db, redis, backend, frontend
REQ-NF-007	로컬 개발은 외부 의존 없이	local 프로파일 H2 + memory 블랙리스트
REQ-NF-008	비밀번호 평문 저장 금지, 로그아웃 후 토큰 무효	bcrypt, jti 블랙리스트 (Redis/memory)

7. Use-Case

7.1 Actor

Actor	설명	진입
설비 엔지니어 (ENGINEER)	부품 교체 요청을 등록하고 AI 결과를 검토·수정해 제출	E_01 /home
안전관리자 (SAFETY_MANAGER)	제출된 요청을 검토해 승인·거절. 최종 책임자	S_01 /manage/requests
AI 검증 에이전트 (System)	규격·법령·안전서류 3종 초안을 생성. 스스로 결정하지 않음	—
인증 시스템 (System)	회원가입·로그인·역할 검사	—

7.2 Use-Case 목록

ID	Use-Case	주 Actor	화면	핵심 API	AI
UC-00	로그인	공통	C_00	POST /auth/login	—
UC-01	회원가입	공통	C_01	POST /auth/signup	—
UC-02	내 요청 현황 확인	엔지니어	E_01	GET /dashboard/summary, GET /work-requests?mine	—
UC-03	교체 요청 등록	엔지니어	E_02	POST /work-requests, POST .../photos	—
UC-04	AI 검증 실행	엔지니어 → AI	E_02→E_03	POST /agent-runs, GET /agent-runs/{id}	AI
UC-05	AI 결과 검토·수정	엔지니어	E_04	PATCH /agent-results/{id}	AI
UC-06	승인 요청 제출	엔지니어	E_04	PATCH .../submit-approval	—

ID	Use-Case	주 Actor	화면	핵심 API	AI
UC-07	내 요청 목록·거절 사유 확인	엔지니어	E_05	GET /work-requests?mine&status	—
UC-08	거절 건 수정 후 재 제출	엔지니어	E_05→E_04	PATCH /agent-results, PATCH .../submit-approval	—
UC-09	승인 대기 요청 관리	안전관리자	S_01	GET /dashboard/summary?role=safety, GET /work-requests?status=PENDING	—
UC-10	요청 상세 검토	안전관리자	S_02	GET /work-requests/{id}, GET .../photos	—
UC-11	승인	안전관리자	S_02	POST /approvals {APPROVE}	—
UC-12	거절 (사유 필수)	안전관리자	S_02	POST /approvals {REJECT, reason}	—

관계: UC-03 →include→ UC-04 →include→ UC-05 →include→ UC-06 (등록은 AI 검증을 포함하고, 검증은 검토를, 검토는 제출을 포함). UC-08 →extend→ UC-07 (거절 사유 확인은 재제출로 확장). UC-09 →include→ UC-10.

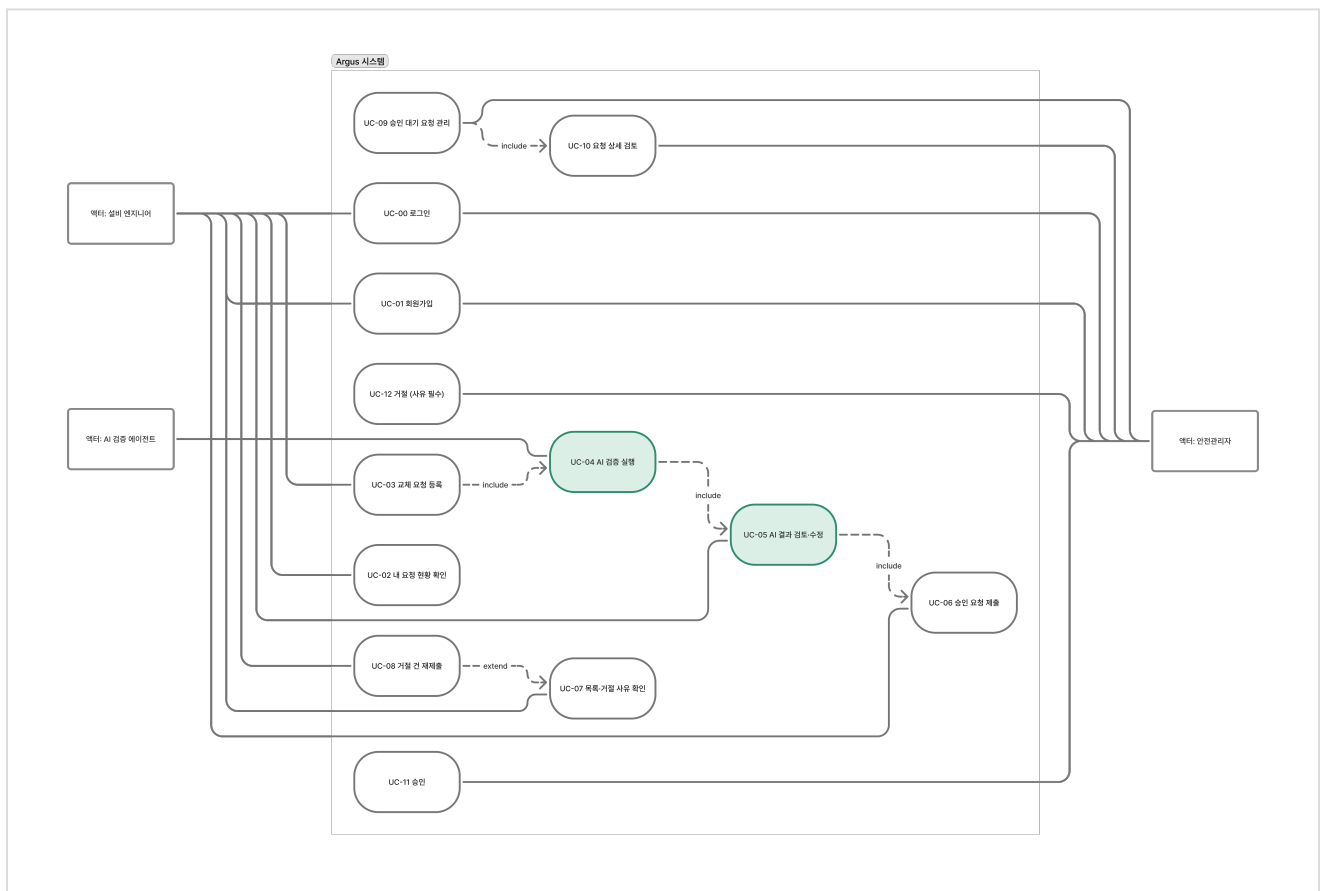


그림 7-1. Use-Case 다이어그램 (FigJam). 초록색이 AI 확장 지점, 점선이 include/extend

7.3 핵심 Use-Case 상세

UC-04 AI 검증 실행 AI 확장 지점

사전조건	요청 필수값 완비, 진행 중 run 없음
------	------------------------

기본 흐름	① POST /agent-runs {workRequestId} → ② 서버가 요청 전체 컨텍스트(설비·라인·물질·운전 조건·제품명·유형·스펙·사진 메타)로 input_snapshot 구성 → ③ agent_runs 1행 + agent_steps 3행(WAITING), 상태 AI_RUNNING → ④ 202 → E_03 → ⑤ 2.5초 폴링 GET /agent-runs/{runId} → ⑥ A1→A2→A3 순 DONE, 카드 갱신 → ⑦ allDone:true → 상태 AI_DONE, [결과 확인] 활성화
AI 확장 지점	⑥. 현재 Mock은 폴링 호출마다 step 하나를 DONE으로 전이하며 고정 결과 반환. 실제 결합 시 오케스트레이터가 3개 에이전트를 백그라운드 실행. 폴링 계약은 불변
예외	진행 중 run 409 RUN_ALREADY_IN_PROGRESS / 필수 미완 400 / step 실패 → 해당 카드 FAILED, HTTP 200 유지
사후조건	agent_results 3행 (수정본 = 원본 복사), 상태 AI_DONE
가드레일	AI는 "제안"만. 조문은 검색 결과 인용, 생성 금지. 근거 없는 항목은 비움

UC-05 AI 결과 검토·수정 AI 확장 지점

사전조건	상태 AI_DONE 또는 REJECTED
기본 흐름	① E_04 진입 → ② A1·A2·A3 결과 표시(편집 가능) → ③ A1 항목 추가·수정 / A2 법령 추가·삭제·편집 / A3 문서 본문 편집 → ④ PATCH /agent-results/{id} (배열 전체 치환) → ⑤ edited:true
확장	원본(original_json) vs 수정본 diff → 엔지니어가 AI 제안을 얼마나 채택했는지 추적 → 에이전트 별 신뢰도 지표 (Phase 2)
예외	PENDING·APPROVED 상태에서 수정 409 RESULT_LOCKED / 타인 요청 403

UC-12 거절 (사유 필수)

사전조건	상태 PENDING, role = SAFETY_MANAGER
기본 흐름	① "거절" → ② 사유 카테고리 + 사유 텍스트(10자 이상) → ③ POST /approvals {REJECT, reason, reasonCategory} → ④ approvals 1행 + 상태 REJECTED (한 트랜잭션) → ⑤ 201
예외	사유 미입력 400 REJECT_REASON_REQUIRED (FE에서 먼저 막고 서버도 검증) / 이미 결정 409 / 엔지니어 호출 403
사후조건	사유가 E_05에 그대로 표시. reason_category 가 S_01 TOP5 집계. UC-08 재제출 가능

7.4 상태 ↔ Use-Case

상태	만드는 UC	다음 가능한 UC
DRAFT	UC-03 임시 저장	UC-03 이어쓰기 → UC-04
AI_RUNNING	UC-04	(폴링 대기)
AI_DONE	UC-04 완료	UC-05, UC-06
PENDING	UC-06, UC-08	UC-10 → UC-11 / UC-12
APPROVED	UC-11	종결
REJECTED	UC-12	UC-07 → UC-08

8. 화면 정의

화면정의서 v2.0(REQ-F-0001, 9화면) 요약. 각 화면의 Acceptance Criteria(Given/When/Then/API)는 원본 문서에 있다.

Screen ID	화면	Role	Path	핵심 요소	호출 API
WRA_C_00	로그인	공통	/login	이메일·비밀번호, 역할별 분기, 401 인라인 에러	2
WRA_C_01	회원가입	공통	/signup	이름·이메일·비밀번호·확인·역할 토글	1
WRA_E_01	엔지니어 메인	엔지니어	/home	KPI 4 (작성중·AI검증중·승인대기·반려), 최근 요청 5, 상태별 이동	4, 6, 7
WRA_E_02	요청 등록	엔지니어	/requests/new	설비·라인·물질·운전조건, 제품명·유형, 유형별 동적 스펙, 사진 업로드, 증상·메모, 임시저장 / AI 검증 시작	5, 8, 9, 11
WRA_E_03	AI 검증 진행	엔지니어	/requests/:id/run	에이전트 카드 3 (대기→실행중→완료), 2.5초 폴링, 완료시 [결과 확인]	12, 7
WRA_E_04	AI 결과 확인·수정	엔지니어	/requests/:id/result	A1·A2·A3 인라인 편집, 법령 추가·삭제, 엔지니어 설명, 제출 전 확인 체크, 제출(422 표시)	7, 13, 8, 14
WRA_E_05	내 요청 목록	엔지니어	/my/requests	상태 탭, 거절 사유 박스, 상태별 이동, 재제출	6, 7, 14
WRA_S_01	요청 관리	안전관리자	/manage/requests	KPI 4 (승인대기·오늘처리·이번달 승인·거절), 대기 테이블, 거절 사유 TOP5, 빈 상태 안내	4, 6
WRA_S_02	요청 상세·승인/거절	안전관리자	/manage/requests/:id	2열: 요청 정보·사진·AI 결과(읽기 전용, 수정 배지)·설명 / Sticky 승인 패널 (승인·거절·사유)	7, 10, 15

8.1 User Flow

엔지니어: C_00 로그인 → E_01 홈 → E_02 등록 → [AI 검증 시작] → E_03 진행(폴링) → E_04 결과 수정·설명 → [제출] → E_05 내 요청
↑ REJECTED 시 사유 확인 → E_04 재진입 → 재제출
안전관리자: C_00 로그인 → S_01 큐 → S_02 상세 → [승인] 또는 [거절 + 사유] → S_01 복귀 (KPI 갱신)

8.2 v1.1 → v2.0 변경 사항 (기획 결정과의 연결)

변경	이유	영향
평균 승인 소요시간·에이전트 진행률 컬럼 제거	3일 범위에서 신뢰할 수 있는 수치를 만들기 어려움. 승인 소요시간은 Phase 2 KPI	대시보드 API 응답 축소
AI 결과물을 엔지니어가 수정 가능	"AI는 제안, 사람이 확정" 원칙. 수정 이력이 채택률 데이터가 됨	agent_results 원본/수정본 분리, PATCH API
현장 사진 → 제품 사진	팩 현장 촬영 제한. 제품 사진은 규격 확인 근거	사진 메타만 AI 입력
제품 유형별 스펙 필드	밸브(압력 등급), 피팅·튜브(연결 규격·재질), 레귤레이터(압력 등급), 필터(물질 종류)가 각각 다름	spec_json 유형별 스키마 검증
안전관리자 체크리스트 blocking 제거	PoC 범위 축소. 승인은 즉시, 거절만 사유 필수	409 게이트 단순화

9. AI-Ready 설계

채점 기준은 AI 정확도가 아니라 "AI가 들어올 자리가 흐름상 타당한가, 프롬프트와 입출력 JSON 스키마가 기존 웹 구조와 호환되는가"다. 이 장은 확장 지점 3곳, 각 지점의 입력·프롬프트 요지·출력 스키마·가드레일, Mock → 실제 전환 방법, AI-Ready 4원칙 대응을 정리한다.

9.1 확장 지점

공통 입력 (input_snapshot): 설비·라인·물질·운전조건(온도·압력)·제품명·제품 유형·spec_json·사진 메타·증상·현장 메모·request_no. 프론트는 workRequestId 만 보내고 서버가 요청 전체로 스냅샷을 구성한다(명세 5.11). 세 에이전트가 같은 스냅샷을 받는다.

지점	Use-Case	출력 (payload_json)	지금 (Mock)	실제 결합 시
A1 규격·호환	UC-04	{ items: [{itemId, text, edited}] }	입력 스펙 기반 고정 판정 (규격 적합, 대체 호환)	LLM이 스펙·운전조건 대조, 대체품 제안. 호환 판정은 호환표 조회 후 설명만 (Phase 2)
A2 법령·조문	UC-04	{ items: [{itemId, text, edited}] }	샘플 조문 3건	법령 인덱스(사전 적재) 검색 → 조문 인용. 생성 금지
A3 안전서류	UC-04	{ documents: [{docId, type, name, content, edited}] } — WORK_PERMIT, RISK_ASSESSMENT 2건 고정	템플릿 고정 문서 2종	LLM이 요청 컨텍스트로 초안 작성, 채울 근거 없는 필드는 비우고 누락 표시
채택률 추적	UC-05	original_json vs payload_json — edited 플래그, 항목별 edited	플래그만	에이전트별 제안 채택률 → 신뢰도 지표

세 지점 모두 "제안"까지다. 제출은 엔지니어(UC-06), 결정은 안전관리자(UC-11/12)가 한다. AI가 판단하지 않는 것은 설계로 강제된다 — 승인 API는 AI 결과를 입력으로 받지 않는다.

9.2 프롬프트 설계 요지

에이전트	역할 지시	가드레일	출력 규칙
A1 규격·호환	반도체 팹 설비 부품의 규격 검토 보조. 요청의 제품 유형·스펙·운전조건을 대조해 적합 여부와 근거를 항목으로 나열	호환 여부는 단정하지 않고 "확인 필요"로 표기. 유독가스 사용 가능 여부 같은 안전 판정은 호환표(Phase 2)가 결정, LLM은 설명만	items 배열, 항목당 한 문장, 근거 없는 항목은 만들지 않음
A2 법령·조문	취급 물질·설비 유형·작업 성격에 적용되는 법령 조문을 검색 결과에서 골라 인용	조문 번호 필수, 없으면 [확인 필요]. 조문 텍스트 생성 금지. 작업 전/중/후 단계 표기	items 배열, "법령명 제 N조 — 요지 (단계)"
A3 안전서류	작업허가서·위험성평가서 초안을 요청 컨텍스트로 채움	채울 근거 없는 필드는 비우고 누락으로 표시. 추정으로 채우지 않음. 문서 2건 고정, 추가 생성 금지	documents 배열, type·name 고정, content 마크다운

프롬프트 전문·스키마 파일·Playground 검증 캡처는 docs/05_ai_ready/prompts.md, schemas/*.schema.json, playground_validation.md 에 있다. 스키마 검증 스크립트 validate_schemas.py 가 Mock 출력과 역예제 13종을 검사한다.

9.3 출력 JSON 스키마 (API 응답 `agentRun.results[]` 과 동일)

```
// A1 · A2 (항목형) - agent_results.payload_json
{ "items": [
  { "itemId": "i-01", "text": "규격 적합: 3000 psi >= 요구 2500 psi", "edited": false },
  { "itemId": "i-02", "text": "대체 호환: SS-8-VCR-2", "edited": true } ] }

// A3 (문서형)
{ "documents": [
  { "docId": "d-01", "type": "WORK_PERMIT", "name": "작업허가서 초안", "content": "...", "edited": false },
  { "docId": "d-02", "type": "RISK_ASSESSMENT", "name": "위험성평가서 초안", "content": "...", "edited": false } ] }
```

스키마 루트가 곧 DB 컬럼(`payload_json`)이고 곧 API 응답이다. 변환 레이어가 없다. `$defs.modelOutput` 으로 "LLM이 반환해야 하는 부분"과 "서버가 붙이는 부분(itemId, edited)"을 분리해, 실제 LLM은 text만 만들면 된다.

9.4 Mock → 실제 LLM 전환

단계	내용	바뀌는 것	바뀌지 않는 것
1	AgentEngine 인터페이스 추출, MockAgentEngine 이 구현체	agent 패키지 내부	API·DB·FE
2	FastAPI AgentOrchestrator 서비스 분리(그림 10-2). Spring의 AgentEngine 구현체는 HTTP 클라이언트가 됨. <code>ai_configs.provider=LOCAL_LLM</code>	구현체 1개, 설정 행, 컨테이너 1개	계약 전부
3	법령 조문을 Chroma 벡터 DB에 사전 적재(법제처 Open API), LegalAgent가 검색·인용	Chroma 컨테이너, 적재 스크립트	A2 출력 스키마
4	오케스트레이터가 3개 병렬 실행, 폴링은 DB 상태만 읽음	Mock의 "호출마다 전이" 로직 제거	202 + 폴링 계약
5	채택률 집계 → 대시보드	dashboard 쿼리	테이블

9.5 AI-Ready 4원칙 대응

원칙	가이드 정의	이 설계에서	근거 위치
Interface First	BE가 실제 AI API를 호출하도록 바뀌어도 FE는 Mock 규격 그대로	FE는 <code>agentRun.results[]</code> 스키마만 안다. <code>AgentService · AgentController · DTO</code> 불변, 교체는 <code>MockAgentEngine</code>	9.3, <code>backend/.../agent</code>
Structured Data	AI가 읽고 쓰기 쉬운 JSON 규격을 DB에 사전 반영	<code>payload_json · original_json · input_snapshot · spec_json</code> 이 스키마 그대로 저장. 유형별 스펙은 <code>product_type</code> 키로 검증	11장, <code>schemas/</code>
Asynchronous Pipeline	Pending/Completed 상태 관리가 가능한 엔드 포인트	<code>POST /agent-runs</code> 202 + <code>GET /agent-runs/{id}</code> 폴링 + <code>allDone</code> . step 단위 <code>WAITING/RUNNING/DONE/FAILED</code>	12장, 13장

원칙	가이드 정의	이 설계에서	근거 위치
Security & Config Isolation	API 카·모델 파라미터를 코드와 분리	ai_configs (agent_code, provider, model_name, prompt_version, temperature, max_tokens, egress_allowed, is_active) + .env . 온프레미스 기본 egress_allowed=false	11장 § 8

9.6 AI-Ready 검토 결과 (09-04)

발표 전 docs/05_ai_ready/ 를 명세·ERD·코드와 대조하는 검토를 수행했다(docs/09_qa/ai_ready_review.md). 발견 13건 중 HIGH 3건은 반영했고 나머지는 알려진 한계로 16장에 기록했다.

#	심각도	발견	조치
1	HIGH	ERD·Use-Case 문서가 저장소에 미커밋	커밋
2	HIGH	A3 부분 수정({docId, content} 만 전송) 시 코드가 type · name 유실	merge 처리로 수정 (FE는 전체 필드 전송이라 데모 영향 없음)
3	HIGH	명세 7장 DB 매핑에 ai_configs · input_snapshot · original_json · request_no 누락	명세 갱신
6	MED	Mock A2 2번 항목이 프롬프트 규칙(조문 번호·단계) 위반	문구 수정
4·5·7~13	MED/ LOW	errorMessage DTO, agents 파라미터 무시 문구, 인터페이스 미추출, .env 키 항목 등	16장 알려진 한계

10. 시스템 아키텍처

10.1 구성도 — 현재 PoC

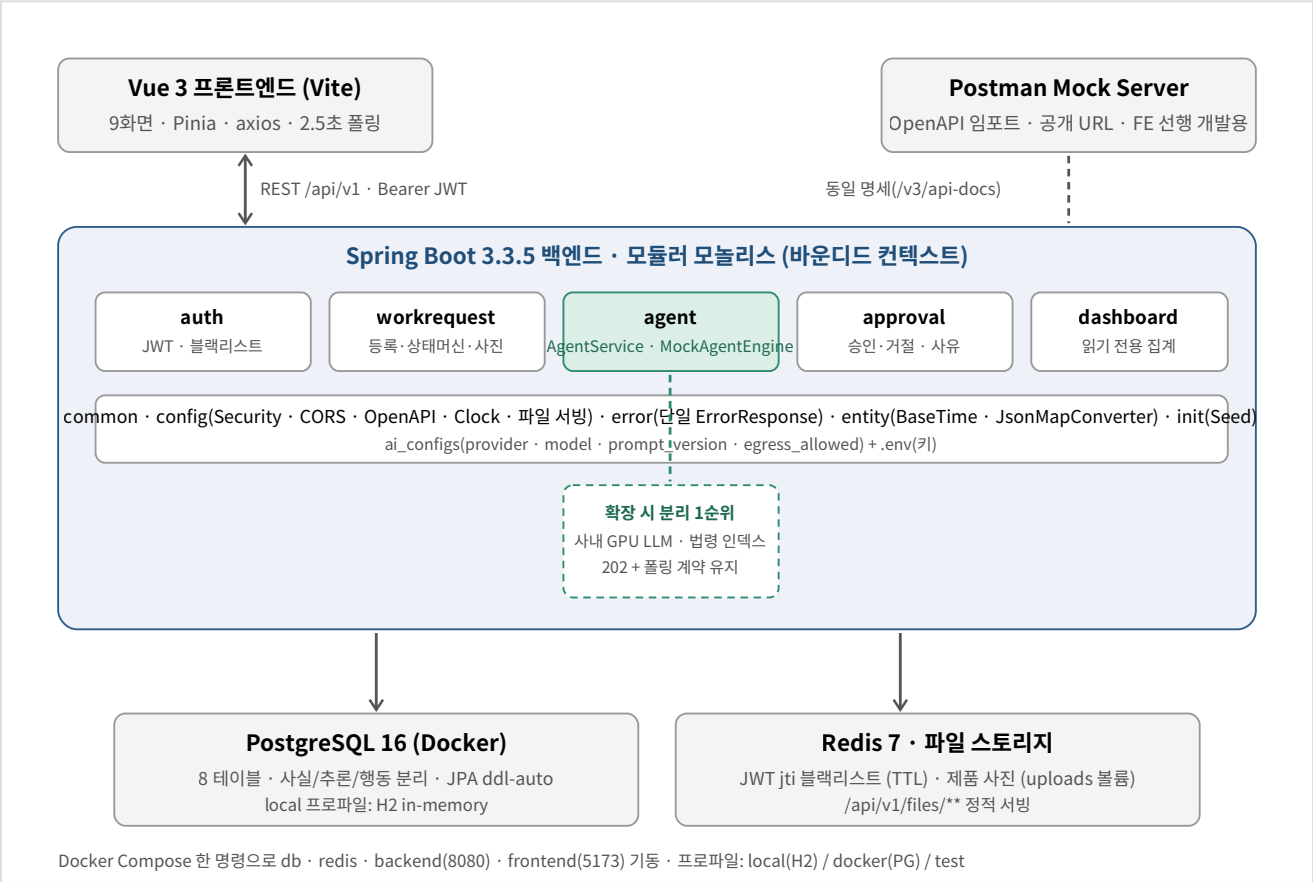


그림 10-1. 현재 PoC 구성. 단일 Spring Boot 프로세스 안에서 agent 컨텍스트가 MockAgentEngine으로 동작. 초록색이 AI 확장 지점이자 확장 시 분리 1순위

10.2 목표 아키텍처 — agent 컨텍스트 분리 후

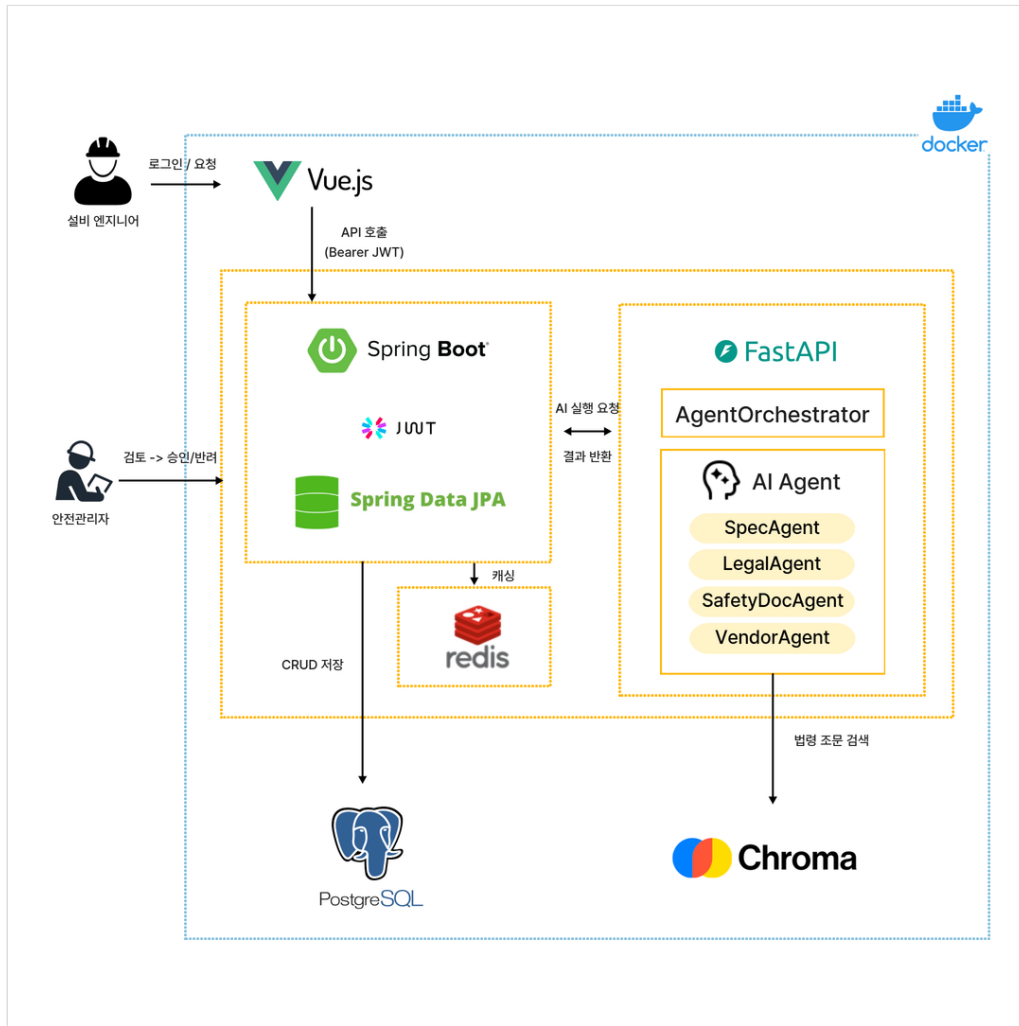


그림 10-2. 실제 LLM 결합 시 목표 구성. Spring Boot(JWT · JPA · Redis · PostgreSQL)는 그대로 두고, agent 컨텍스트만 FastAPI AgentOrchestrator 서비스로 분리해 SpecAgent · LegalAgent · SafetyDocAgent(· VendorAgent, Phase 2)를 실행. 법령 조문은 Chroma 벡터 DB에서 검색. Spring ↔ FastAPI 사이는 현재의 202 + 폴링 계약 그대로

구분	현재 PoC (그림 10-1)	목표 (그림 10-2)
AI 실행	MockAgentEngine — Spring Boot 프로세스 내부, 폴링마다 step 전이	FastAPI AgentOrchestrator 별도 프로세스, 3~4 에이전트 병렬 실행
법령 근거	고정 샘플 조문	Chroma 벡터 DB에 법령 조문 사전 적재 → LegalAgent가 검색·인용
계약 · FE · DB · 승인 흐름	불변 — POST /agent-runs 202 · GET /agent-runs/{id} 폴링 · agent_results.payload_json 스키마 · FE 화면. Spring Boot는 AI 요청을 FastAPI로 위임하고 결과를 같은 테이블에 저장	
왜 FastAPI인가	LLM SDK·벡터 DB 클라이언트·에이전트 프레임워크가 Python 생태계에 있고, GPU 서버에 배포되는 단위가 웹 API와 다르기 때문. 실행 환경·스케일링·변경 주기가 다른 것만 분리한다는 10.3 기준에 부합	

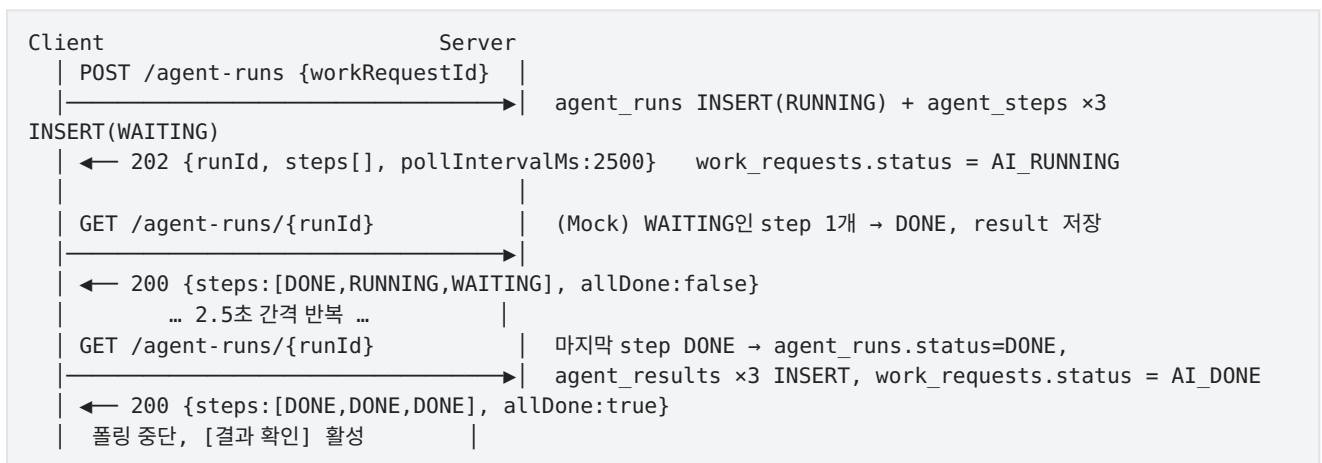
10.3 왜 모듈러 모놀리스인가 (MSA 미채택 근거)

판단 기준	이 프로젝트	결론
배포 단위·팀 구조	서비스 1개, 팀 5명, 3일. 독립 배포·독립 스케일링·기술 스택 분리 필요 없음	MSA의 존재 이유가 해당 없음

판단 기준	이 프로젝트	결론
트랜잭션 경계	AI 검증 시작(runs+steps+status), 3종 완료 (runs+results+status), 승인(approvals+status)이 각각 한 트랜잭션. 서비스를 켜면 전부 분산 트랜잭션	같은 프로세스·같은 DB
타깃 고객	GPU·데이터 전담 조직이 없는 협력사. 마이크로서비스 운영 부담을 엮을 수 없음	단일 배포
분리 기준	규모가 아니라 "실행 환경·스케일링·변경 주기가 다른가"	agent 가 1순위: 온프레미스 LLM·GPU, 긴 실행 시간, 다른 스케일링. 202+폴링 계약이 있어 프로세스 밖으로 빼도 FE·BE 계약 불변 → 그림 10-2의 FastAPI 오케스트레이터. 2 순위 법령 인덱스(Chroma, 갱신 주기 다름)

적용한 것은 전략적 DDD다 — 바운디드 컨텍스트 경계, Aggregate Root(WorkRequest 가 상태를 소유, AgentRun · Approval 은 독립 루트), 유비쿼터스 언어(명세 2장 enum이 FE·BE·API·발표에서 같은 단어). 도메인 이벤트 ·CQRS·핵사고날은 3일 범위에서 오버엔지니어링이라 뺐다. dashboard 만 여러 컨텍스트를 읽는 예외이며 읽기 전용 조회 모델로 명시한다.

10.4 비동기 파이프라인



실제 LLM에서는 오케스트레이터가 백그라운드에서 3개를 병렬 실행하고 폴링은 DB 상태만 읽는다. 큐(Redis Streams·RabbitMQ)는 아키텍처상 "확장 시 도입"이며 3일 범위에서는 스레드 기반이면 충분하다(교수 상담 확인).

10.5 보안

항목	설계
인증	JWT Bearer (jjwt 0.12), /auth/signup · /auth/login · /files/** 외 전 API 필수. 로그아웃 시 jti 블랙리스트 (Redis TTL = 토큰 잔여 수명, local은 memory)
권한	WorkRequestAccessPolicy 한 곳에서 판단: ENGINEER는 본인 요청만(403 FORBIDDEN_NOT_OWNER), SAFETY_MANAGER는 PENDING 이상 전체, 승인 API는 SAFETY_MANAGER만(403 FORBIDDEN_ROLE)
비밀번호	bcrypt 해시만 저장. passwordConfirm 은 저장하지 않음
파일	/api/v1/files/** 는 permitAll (img 태그가 Bearer를 못 붙임). 키가 UUID라 추측 불가. 운영 시 서명 URL로 전환
AI 설정	API 키는 .env, 모델·프롬프트 버전·파라미터는 ai_configs. egress_allowed=false 기본 → 외부 전송 차단이 기본값
입력 검증	Bean Validation + 유형별 spec_json 스키마 검증, 파일 타입·크기·개수 제한(400/413/409)

11. 데이터 모델 (ERD)

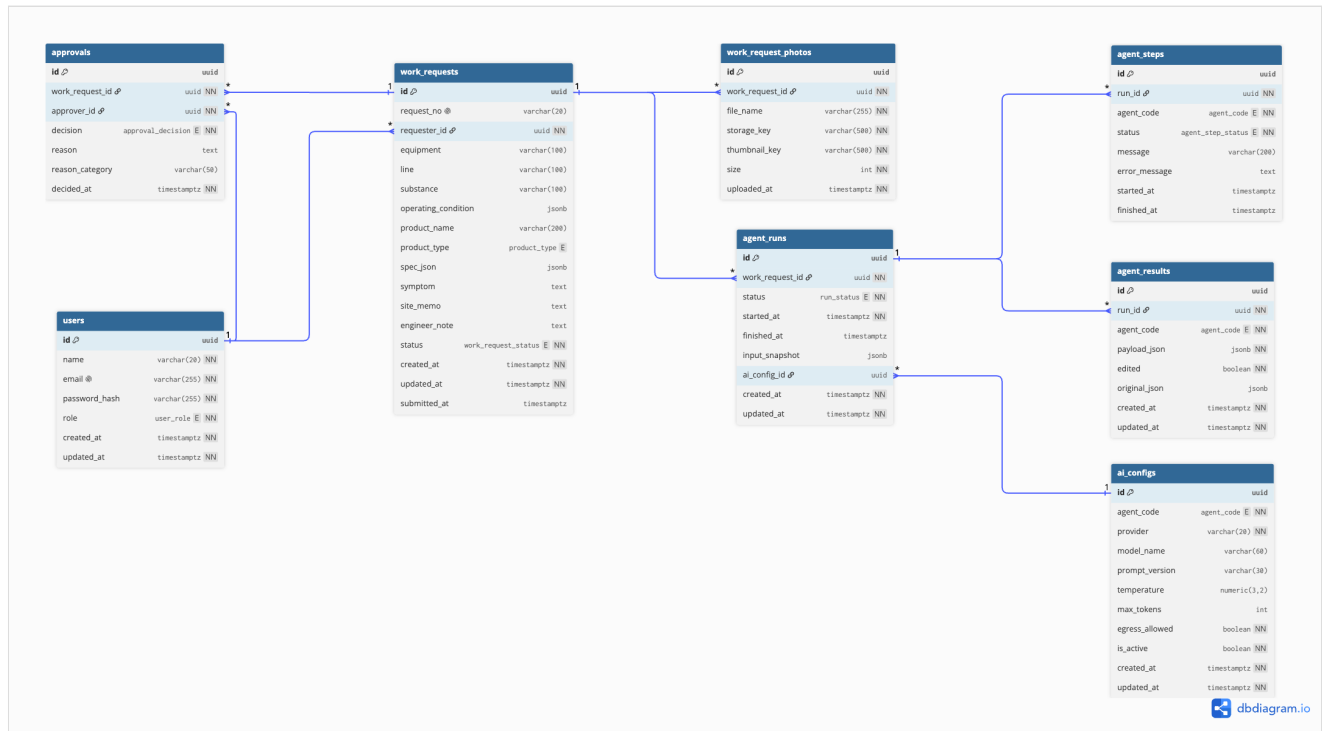


그림 11-1. ERD (dbdiagram.io). 8 테이블, FK 8, enum 7. 코드(엔티티) 기준으로 09-04 대조 완료

11.1 설계 원칙

원칙	적용
대리키 PK + 자연키 UNIQUE	전 테이블 UUID v4 PK. 업무 식별자(<code>users.email</code> , <code>work_requests.request_no</code> , <code>(run_id, agent_code)</code>)는 UNIQUE. 번호·코드 체계가 바뀌어도 FK 무결성 유지, 조인은 대리키로
사실 / 추론 / 행동 분리	입력(<code>work_requests</code> , <code>photos</code>) · AI 산출(<code>agent_runs</code> , <code>agent_steps</code> , <code>agent_results</code>) · 사람의 결정(<code>approvals</code>). 갱신 주체와 주기가 달라 테이블을 나눔
append-only 이력	<code>agent_runs</code> (재실행), <code>approvals</code> (재제출 후 재결정)는 갱신하지 않고 행 추가. 최신 1건을 화면에
원본 보존	<code>agent_results.original_json</code> (AI 원본) vs <code>payload_json</code> (수정본). <code>agent_runs.input_snapshot</code> 에 실행 시점 입력 전체
JSON은 구조가 가변인 곳에만	<code>operating_condition</code> , <code>spec_json</code> (유형별 키), <code>payload_json</code> (에이전트별). 조회·집계 키(<code>status</code> , <code>agent_code</code> , <code>reason_category</code> , <code>edited</code>)는 컬럼
상태는 enum	값이 고정이고 코드에서 분기하는 것은 록업 테이블이 아니라 enum. 확장 대상은 데이터(설비·법령)지 상태값이 아님

11.2 테이블 요약

테이블	역할	핵심 컬럼	제약	API
users	계정	name, email, password_hash, role	email UNIQUE	1·2·3·16

테이블	역할	핵심 컬럼	제약	API
work_requests	교체 요청 · 상 태머 신 주체	request_no, requester_id, equipment, line, substance, operating_condition(json), product_name, product_type, spec_json, symptom, site_memo, engineer_note, status, submitted_at	request_no UNIQUE, requester_id FK. draft 허용을 위해 업무 컬럼은 nullable(서비스 검증)	5·6·7·8·14
work_request_photos	제품 사진	file_name, storage_key, thumbnail_key, size	work_request_id FK. ≤5장 · ≤10MB 서비스 검증	9·10
agent_runs	AI 실행 단위	status, input_snapshot(json), ai_config_id, started_at, finished_at	work_request_id FK, ai_config_id FK. RUNNING 중복은 서비스 409	11·12
agent_steps	에이 전트 별 진행	agent_code, status, message, error_message	(run_id, agent_code) UNIQUE	11·12
agent_results	AI 결과 · 수 정보	agent_code, payload_json, original_json, edited	(run_id, agent_code) UNIQUE. PENDING·APPROVED 시 409	7·12·13
approvals	승인 · 거 절	approver_id, decision, reason, reason_category, decided_at	work_request_id FK, approver_id FK. REJECT 시 reason 필수(서비스)	15·7
ai_configs	에이 전트 설정	agent_code, provider, model_name, prompt_version, temperature, max_tokens, egress_allowed, is_active	활성 1개/에이전트 (시드·서비스)	11·12

전 테이블에 created_at · updated_at (JPA Auditing). enum 7종: user_role, work_request_status(DRAFT · AI_RUNNING · AI_DONE · PENDING · APPROVED · REJECTED), product_type(5), run_status, agent_code(A1 · A2 · A3), agent_step_status(WAITING · RUNNING · DONE · FAILED), approval_decision.

11.3 관계와 정규화

관계	종류	근거
users → work_requests	1:N	엔지니어가 요청 여러 건
users → approvals	1:N	안전관리자가 여러 건 결정
work_requests → work_request_photos	1:N	최대 5장
work_requests → agent_runs	1:N	재실행 시 새 run, 이전 run 보존
work_requests → approvals	1:N	거절 → 재제출 → 재결정 (append-only)
agent_runs → agent_steps / agent_results	1:N (고정 3)	A1·A2·A3
ai_configs → agent_runs	1:N	설정 하나로 여러 실행. 어떤 설정으로 돌았는지 추적

- **2NF**: 사진·실행·결과·승인을 요청 행에 반복 컬럼으로 두지 않고 자식 테이블로 분리.
- **3NF**: agent_steps (진행 상태, 오케스트레이터가 초 단위 갱신)와 agent_results (결과물, 엔지니어가 편집)를 분리. 한 테이블이면 폴링 UPDATE와 편집 UPDATE가 같은 행을 경합.

- **N:M은 이번 범위에 없음.** 법령 마스터(law_index)와 결과의 N:M, 설비 마스터·호환표는 Phase 2(A1 호환표 연동과 함께).
- **PoC 구현 차이:** JSON 컬럼은 H2/PostgreSQL 호환을 위해 text + JsonMapConverter, enum은 varchar + CHECK. ERD의 jsonb·enum 표기는 PostgreSQL 단독 운영 시 목표 타입. 권장 인덱스 5개는 데모 규모에서 미생성.

11.4 상태머신

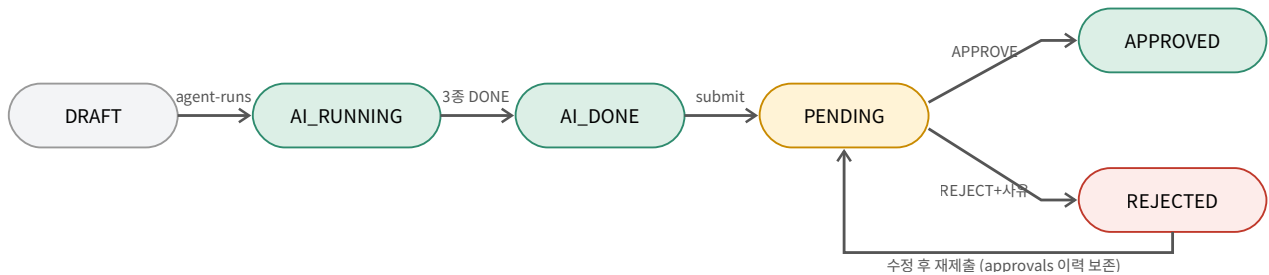


그림 11-2. work_requests.status 전이. 전이 조건·오류 코드는 API 명세 3장·8장

12. API 설계

12.1 공통 규약

Base URL	/api/v1
인증	JWT Bearer. signup·login 제외 전 API 필수
ID · 시각	UUID v4 · ISO 8601 KST 오프셋(2026-09-03T10:22:00+09:00)
페이지네이션	page (0-base) · size (기본 20, 최대 100) · sort=필드,asc desc. 응답 page 객체. 잘못된 sort 400
에러 포맷	{timestamp, status, code, message, path, fieldErrors[]} 단일. fieldErrors는 400~422에만
권한	ENGINEER 본인 요청만 / SAFETY_MANAGER PENDING 이상 전체. 위반 403

12.2 API 목록 (16개, 전부 구현·테스트)

#	Method	Path	설명	화면	권한	코드	주요 오류
1	POST	/auth/signup	회원가입	C_01	공개	201	400 VALIDATION_FAILED · PASSWORD_MISMATCH, 409 EMAIL_ALREADY_EXISTS
2	POST	/auth/login	로그인 · 역할 · redirectPath	C_00	공개	200	401 INVALID_CREDENTIALS
3	GET	/auth/me	토큰 소유자	공통	인증	200	401 TOKEN_EXPIRED · TOKEN_INVALID
4	GET	/dashboard/summary? role=	역할별 KPI + 거절 TOP5	E_01, S_01	인증	200	403 역할 불일치

#	Method	Path	설명	화면	권한	코드	주요 오류
5	POST	/work-requests	요청 생성 (draft / 정식) · requestNo 채번	E_02	ENGINEER	201	400 VALIDATION_FAILED · SPEC_SCHEMA_MISMATCH
6	GET	/work-requests	목록 (mine · status · page · sort) · nextAction	E_01, E_05, S_01	인증	200	400 sort
7	GET	/work-requests/{id}	상세 (사진 · 최신 run 결과 · 최신 approval)	E_04, E_05, S_02	인증	200	403 FORBIDDEN_NOT_OWNER, 404
8	PATCH	/work-requests/{id}	부분 수정 · 엔지니어 설명	E_02, E_04	ENGINEER	200	409 IMMUTABLE_STATUS, 403
9	POST	/work-requests/{id}/photos	제품 사진 (multipart, ≤5, ≤10MB)	E_02	ENGINEER	201	400 UNSUPPORTED_FILE_TYPE, 413, 409 PHOTO_LIMIT_EXCEEDED
10	GET	/work-requests/{id}/photos	사진 목록 · 원본 URL	S_02	인증	200	404
11	POST	/agent-runs	AI 3종 실행 (비동기)	E_02	ENGINEER	202	409 RUN_ALREADY_IN_PROGRESS, 400 WORK_REQUEST_INCOMPLETE
12	GET	/agent-runs/{runId}	진행 폴링 · allDone	E_03	인증	200	404 AGENT_RUN_NOT_FOUND
13	PATCH	/agent-results/{id}	결과 수정 (items / documents 전체 치환)	E_04	ENGINEER	200	409 RESULT_LOCKED, 403
14	PATCH	/work-requests/{id}/submit-approval	제출 → PENDING	E_04, E_05	ENGINEER	200	422 SUBMIT_REQUIRED_FIELD_MISSING, 409
15	POST	/approvals	승인 / 거절 + 사유	S_02	SAFETY_MANAGER	201	400 REJECT_REASON_REQUIRED, 403 FORBIDDEN_ROLE, 409 ALREADY_DECIDED · NOT_PENDING
16	POST	/auth/logout	토큰 무효화 (jti 블랙리스트)	공통	인증	204	401

12.3 RESTful 규격 준수 포인트

- 리소스 명사 복수형, 하위 리소스는 경로로 (/work-requests/{id}/photos), 상태 전이 액션만 동사 경로 (/submit-approval).
- 상태 코드 의미 분리: 생성 201, 비동기 수락 202, 입력 오류 400, 인증 401, 권한 403, 없음 404, 상태 충돌 409, 용량 413, 검증 실패 422.

- 같은 409라도 code 로 원인 구분(RUN_ALREADY_IN_PROGRESS / IMMUTABLE_STATUS / RESULT_LOCKED / ALREADY_DECIDED / NOT_PENDING / PHOTO_LIMIT_EXCEEDED / EMAIL_ALREADY_EXISTS).
- 역할별 응답 차이는 필드 값으로(editable:false) 표현하고 엔드포인트를 나누지 않음.

12.4 Mock API

계층	무엇	용도
백엔드 내 Mock	MockAgentEngine — AI 확장 지점만 Mock. 폴링 호출마다 step 하나 DONE, 고정 JSON	실제 데모. 나머지 15 API는 실제 DB
Swagger	/swagger-ui.html, /v3/api-docs. Bearer 인증 스킴, Pageable 분리	명세 열람·즉시 호출
Postman Mock Server	OpenAPI 임포트 → 컬렉션 → 공개 Mock (*.mock.pstmn.io). 16 요청 예시 응답	백엔드 없이 FE 개발·명세 검증. URL은 docs/07_api/README.md

13. 데이터 흐름과 핵심 트랜잭션

13.1 화면 → API → 저장 요약

단계	화면	API	INSERT	UPDATE	응답
회원가입	C_01	1	users	—	201
로그인	C_00	2	—	—	200 + token · role · redirectPath
요청 등록 (임시)	E_02	5 draft:true	work_requests (DRAFT, request_no 채번)	—	201
사진 업로드	E_02	9	work_request_photos	—	201
AI 검증 시작	E_02	11	agent_runs (RUNNING, input_snapshot) + agent_steps × 3 (WAITING)	work_requests.status = AI_RUNNING	202
폴링 (Mock)	E_03	12	agent_results (step DONE 시, original=payload)	agent_steps.status · message	200 allDone:false
3종 완료	E_03	12 (마지막)	—	agent_runs.status=DONE · finished_at, work_requests.status = AI_DONE	200 allDone:true
결과 수정	E_04	13	—	agent_results.payload_json · edited	200
설명 저장	E_04	8	—	work_requests.engineer_note	200
제출	E_04	14	—	work_requests.status = PENDING · submitted_at	200 / 422
승인 · 거절	S_02	15	approvals	work_requests.status = APPROVED / REJECTED	201
재제출	E_04	14	—	status = PENDING · submitted_at 갱신 (approvals 이력 보존)	200

13.2 한 트랜잭션이어야 하는 세 곳

트랜잭션	류이는 쓰기	깨지면
AI 검증 시작	agent_runs INSERT + agent_steps ×3 INSERT + work_requests.status	run 없이 상태만 AI_RUNNING → 폴링 대상이 없음
3종 완료	마지막 step DONE + agent_runs DONE + agent_results ×3 + work_requests.status=AI_DONE	결과 없이 AI_DONE → E_04가 빈 화면
승인 · 거절	approvals INSERT + work_requests.status	결정 기록 없이 상태만 바뀜 → 감사 불가

이 세 트랜잭션이 "왜 서비스를 쪼개지 않았나"의 근거다(10.2).

14. 구현·검증 현황 (09-04 기준)

14.1 구현 범위

영역	상태	근거
백엔드 API 16개	전부 구현 · 통합 테스트	기준 커밋 62a9def (origin/main), main 소스 91개, 테스트 10개
통합 테스트	38 통과 (H2) + 스키마-문서 정합성 테스트 1	<code>./gradlew clean build</code> , <code>SchemaDocConsistencyTest</code>
Docker	4 컨테이너 정상, 시드 자동 적재	<code>down -v → up -d --build</code> , 로그 <code>users=3</code> , <code>workRequests=7</code> , <code>agentRuns=6</code> , <code>photos=2</code> , <code>approvals=5</code>
Swagger · Postman	/v3/api-docs 200, Pageable 분리, 공개 Mock 서버	<code>docs/07_api/README.md</code>
프론트엔드	스캐폴딩 + 핵심 화면 (로그인 → 목록 → 상세 → 승인)	<code>frontend/src/views/</code>
ERD ↔ 코드	8 테이블 컬럼 수 전부 일치, DBML 문법 검증 통과	Hibernate SchemaExport DDL, 대조표 (<code>docs/06_erd/</code>)
AI-Ready 문서 ↔ 코드	Mock 출력이 스키마 4종 통과, 역예제 13종 검사	<code>docs/05_ai_ready/validate_schemas.py</code> , 검토 보고서

14.2 End-to-End 검증 (PostgreSQL, curl)

```
로그인(ENGINEER) → POST /work-requests → 201 requestNo=WR-20260903-001
→ POST .../photos (PNG) → 201, /api/v1/files/... 무인증 200 image/png
→ POST /agent-runs → 202 RUNNING
→ GET /agent-runs/{runId} ×3 → [DONE, WAITING, WAITING] → [DONE, DONE, WAITING] → [DONE, DONE, DONE]
allDone=true
→ GET /work-requests/{id} → status AI_DONE, agentRun.results 3종
→ PATCH .../submit-approval (설명 없음) → 422 fieldErrors[engineerNote] → (설명 포함) → 200 PENDING
→ 로그인(SAFETY_MANAGER) → GET .../{id} → editable=false → POST /approvals APPROVE → 201 APPROVED
→ 로그아웃 → 같은 토큰 401 TOKEN_REVOKED (Redis auth:blacklist:{jti} TTL 3599)
```

시드 데이터로 상태별 진입도 검증했다: DRAFT(사진 2장) · AI_RUNNING(1/3, 폴링 이어서) · AI_DONE(제출 가능) · PENDING · REJECTED(거절 이력 3건) · APPROVED. 라이브 데모가 중간에 막혀도 시드 상태에서 이어서 시연할 수 있다.

14.3 검증 도구별 결과

도구	대상	결과
Gradle 통합 테스트	auth · dashboard · 요청 흐름(등록→AI→폴링→수정→제출→승인) · 필수값/draft · 스펙 스키마 · 사진 제한 · 블랙리스트	38 passed
Redocly lint	openapi.yaml	valid
@dbml/cli dbml2sql	fixguide_erd.dbml	CREATE TABLE 8 · FK 8 · TYPE 7
validate_schemas.py	A1·A2·A3 Mock 출력 + 역예제 13	통과
Hibernate SchemaExport ↔ DBML ↔ erd.md	8 테이블 컬럼 수	전부 일치

15. 결정 기록

ID	일자	결정	이유	버린 대안
D-01	09-02	"SK AX AI 도메인팀이 고객사에 제안한다" 프레임	페르소나·문제 정의가 선명해짐. 교수 상담에서 확인. 제안서 형식은 발표 앞 3분에만	일반 서비스 기획
D-02	09-02	반도체 협력사 설비 도메인 (E안) 채택	하이닉스 자율형 팹 → 협력사 확산 스토리, SK AX 공백 영역, 팀원 도메인 지식	N안 저축은행 불완전판 매 검수 (금융)
D-03	09-02	문제를 "알람 분석"이 아니라 "조치 이후 변경 승인"으로	알람 분석·AR 가이드는 상용화됨 (AMOS, RealWear, 버넥트). 승인 병목은 확인된 솔루션 없음. 못 줄이는 것(승인 자체·외부 검사) 명시	알람 → 원인 → 정비 가이드
D-04	09-02	AI 확장 지점 3곳 (A1 규격·A2 법령·A3 안전 서류). A4 벤더·A1 호환표 연동은 Phase 2	3일 범위. 화면정의서 v2.0 기준. 유독 가스 호환 같은 안전 판정은 LLM이 아니라 호환표가	에이전트 4개 + 오케스트레이터 타임라인
D-05	09-02	모듈러 모놀리스 + 바운디드 컨텍스트, MSA 미채택	트랜잭션 경계 3개, 협력사 운영 부담. agent가 분리 1순위, 202+폴링 계약 유지	MSA (Eureka·Gateway)
D-06	09-03	화면정의서 v2.0: 체크리스트 blocking 제거, 결과 수정 가능, 제품 사진, 유형별 스펙, 승인시간 KPI 제거	범위 축소와 "AI는 제안" 원칙. 3일 안에 신뢰할 수치 불가	v1.1
D-07	09-03	Spring Boot 3.3.5 com.skala.fixguide 구조 채택, 초기 스캐폴드(Boot 4.1) 폐기, 다른 PR을 포팅 병합	실제 구현이 먼저 나온 쪽을 채택. 팀원 커밋은 조상으로 보존	초기 스캐폴드 유지 후 재작성
D-08	09-03	대리키 UUID PK + 자연키 UNIQUE, requestNo 신설	번호 체계 변경에 FK 무결성 유지. 화면엔 업무 번호, URL·FK는 UUID	업무 번호를 PK로
D-09	09-03	AI_DONE 상태 신설, 제출 경로 /work-requests/{id}/submit-approval, 결과 수정은 전체 치환	E_03/E_04 구분, 리소스 식별, 추가·삭제·편집을 한 호출로	run 조인으로 구분, 항목 단위 API

ID	일자	결정	이유	버린 대안
D-10	09-03	PostgreSQL 16 Docker (데모) + H2 (테스트·로컬), Supabase 미사용	강의실 네트워크 리스크, 온프레미스 스토리 일치, jsonb 전환 로드맵	Supabase 클라우드
D-11	09-03	회원가입 포함 (당초 제외)	역할 선택·데모 설득력. 비용 2시간	시드 계정만
D-12	09-04	request_no DB nullable 유지, 권장 인덱스 5개 미생성	NOT NULL 전환은 전원 볼륨 재시드 필요. 데모 규모에서 인덱스 영향 없음	NOT NULL + 인덱스
D-13	09-04	Postman Mock 서버 추가 (백엔드 완성 후)	루브릭 항목. 실제 데모는 백엔드	생략
D-14	09-04	A3 "항목 추가" 버튼 유지, 데모 대본에서 사용 금지 + 한계로 명시	제거 비용 대비 발표 시간	버튼 제거

16. 리스크·한계·로드맵

16.1 알려진 한계 (정직하게)

#	한계	영향	대응
1	AI는 전부 Mock. 폴링 호출마다 step 하나가 DONE되는 상태머신	실제 품질 미검증	채점 기준(Mock)에 부합. 전환 지점 MockAgentEngine 하나
2	A3 문서 "항목 추가" 시 type 미지정 → 스키마(2건 고정) 위반 가능	데모에서 누르면 비정형 문서	대본에서 제외. Phase 2에 타입 선택 UI
3	FAILED step의 errorMessage가 DTO에 없음, Mock은 FAILED를 만들지 않음	명세 5.12와 불일치	Phase 2
4	명세 5.11 agents 배열을 서비스가 무시하고 항상 3종 실행	부분 실행 불가	명세에 PoC 동작 명시
5	JSON 컬럼이 text, enum이 varchar	jsonb 연산·인덱스 불가	PostgreSQL 단독 운영 시 전환
6	RequestNoGenerator 동시 요청 시 중복 가능	데모 규모에선 무관	시퀀스 또는 행 잠금
7	ddl-auto: update, 테스트는 H2(MODE=PostgreSQL)	PG 전용 쿼리 미검증 가능성	PG 수동 E2E로 현재 문제 없음. Flyway·Testcontainers
8	썸네일이 원본과 동일 키, EXIF 제거 미구현	대역폭	Phase 2
9	테스트 공백: 사진 저장 경로, 타 엔지니어 요청 403, REJECTED→재제출→재승인 이력	커버리지	발표 후 추가
10	설비·라인이 문자열, 부품 마스터·호환표 없음	정규화 여지	Phase 2 A1 호환표 연동과 함께 마스터 분리

16.2 발표·데모 리스크

리스크	대응
강의실 네트워크	전부 로컬 Docker. Postman Mock은 보조
라이브 흐름 중단	시드 7건이 상태별로 존재 → 어느 단계에서든 이어서 시연. 노트북 2대, 백업 영상
"사내 에이전트로 되지 않나"	법령 검색은 된다. 다른 건 요청 컨텍스트·사내 절차를 결합한 분류가 승인 큐·이력으로 이어지는 워크플로우, 원본·수정본 비교로 쌓이는 채택률, 그리고 협력사는 외부 LLM을 못 쓴다는 것

리스크	대응
"승인 기간이 얼마나 줄어드나"	외부 검사 대기는 못 줄인다. 서류 준비·반려 왕복만 대상이고 비중은 현업 확인 필요
"AI가 승인하냐"	아니다. 분류·초안·누락 표시까지. 승인 API는 AI 결과를 입력받지 않는다
"법령을 잘못 인용하면"	조문은 검색 결과 인용, 생성 금지. 없으면 [확인 필요]. 실제 결합 시 인용 조문 존재를 코드로 검증

16.3 로드맵

단계	내용	바뀌는 것
Step 1 — 실제 LLM	AgentEngine 인터페이스 → FastAPI AgentOrchestrator 서비스(그림 10-2), 사내 GPU LLM, <code>ai_configs.provider=LOCAL_LLM</code> , 에이전트 병렬 실행	agent 패키지 + 컨테이너 1개
Step 2 — 법령 RAG	법제처 Open API 사전 적재 → Chroma 벡터 DB, LegalAgent가 조문 검색·인용, 인용 존재 검증	Chroma 컨테이너, 적재 스크립트
Step 3 — 신뢰도 추적	원본 vs 수정본 diff → 에이전트별 채택률 KPI, 협력사별 프롬프트 튜닝	dashboard 쿼리
Step 4 — 마스터·호환	설비·부품 마스터, 호환표(<code>part_compatibility</code>), A1이 호환표 조회 후 설명	테이블 3, A1 구현체
Step 5 — 확산	멀티테넌트(협력사별 설정), agent 컨텍스트 분리 배포, 현장 확인 채널(폰 카메라 태그 스캔·사진 첨부)	tenants 테이블, 배포 단위

17. 평가 루브릭 대응표

17.1 정량 Rubric (60%)

평가 영역	세부 기준	이 문 서	저장소 산출물
서비스 기획 & Architecture (30)	Use-Case 정의 및 UI 와이어프레임(Figma) 완성도	7장, 8장	<code>docs/02_usecase/usecase_spec.md</code> , FigJam 다이어그램, <code>docs/03_wireframe/</code> , 화면정의서 v2.0
서비스 기획 & Architecture (30)	AI 확장 지점 정의 및 프롬프트/JSON 스키마 타당성	9장	<code>docs/05_ai_ready/prompts.md</code> , <code>schemas/</code> 4종, <code>playground_validation.md</code> , <code>validate_schemas.py</code>
서비스 기획 & Architecture (30)	GitHub 레포지토리 관리 및 R&R 분담의 적절성	4장, 5장	브랜치·PR 규칙, 병합 이력, 각자 이름 커밋, <code>docs/01_planning/</code>
서비스 기획 & Architecture (30)	FE-BE-DB 전체 시스템 구조 다이어그램의 명확성	10장	<code>docs/04_architecture/</code>
시스템 설계 및 Scaffolding (30)	ERD 테이블 관계(1:N, N:M) 및 정규화 타당성	11장	<code>docs/06_erd/erd.md</code> , <code>fixguide_erd.dbml</code> , <code>schema_postgres.sql</code> , <code>erd.svg</code>
시스템 설계 및 Scaffolding (30)	Mock API 엔드포인트 구성 완성도 및 RESTful 규격 준수	12장	<code>docs/07_api/README.md</code> , Swagger, Postman 컬렉션·Mock 서버
시스템 설계 및 Scaffolding (30)	FE/BE 프로젝트 구조 및 DB 연동 정상 여부	10장, 14장	<code>backend/</code> 91 소스 · 38 테스트, <code>frontend/</code> , <code>docker-compose.yml</code>

평가 영역	세부 기준	이 문 서	저장소 산출물
시스템 설계 및 Scaffolding (30)	Mock API를 활용한 실제 데이터 바인딩 및 화면 시연	13장, 14장	E2E 검증 보고서(캡처 10장), 시드 데이터, 데모 대본

17.2 Peer Review (40%)

평가 영역	세부 기준	대응
기획 및 UX	문제 정의(페르소나)가 명확한가	2장: 두 페르소나, As-Is 7단계 소요, 못 줄이는 것 명시
기획 및 UX	와이어프레임·User Flow가 직관적인가	8장: 9화면, 상태별 이동 규칙을 서버가 <code>nextAction</code> 으로
시스템 설계	ERD 관계·정규화가 적절한가	11장: 사실/추론/행동 분리, 원본 보존, 대리키·자연키
시스템 설계	REST API 명세가 체계적인가	12장: 16 API, 상태 코드 의미 분리, 단일 예외 포맷
AI-Ready 확장성	확장 지점이 서비스 흐름상 타당한가	7장 UC-04·05, 9.1: 제안까지만, 결정은 사람
AI-Ready 확장성	프론트·JSON 스키마가 기존 웹 구조와 호환되는가	9.3: 스키마 = DB 컬럼 = API 응답, 변환 레이어 없음
구현 및 Pitch	Mock API 기반 로컬 데모가 안정적인가	14장: Docker 한 명령, 시드 7건 상태별, 백업 영상
구현 및 Pitch	15분 발표가 논리적인가	가이드 6섹션 배분, 리허설 2회, 예상 질문 6개(16.2)

부록

A. 실행 방법

```
git clone https://github.com/Taeyum/skala-fixguide && cd fixguide
cp .env.example .env
docker compose up -d --build          # db(PostgreSQL 16), redis, backend :8080, frontend :5173
# Swagger: http://localhost:8080/swagger-ui.html
# 로컬(H2)만: cd backend && ./gradlew bootRun
# 테스트: ./gradlew clean build
# 기존 볼륨에 requestNo가 없으면: docker compose down -v 후 재기동
```

B. 시드 데이터

계정 (비밀번호 공통 Passw0rd!23)	역할
engineer@fixguide.dev (이엔지)	ENGINEER
engineer2@fixguide.dev (김현장)	ENGINEER
safety@fixguide.dev (박안전)	SAFETY_MANAGER

상태	설비	같이 들어가는 것	데모 용도
DRAFT	펌프 P-114	사진 2장	이어쓰기 → AI 검증 시작
AI_RUNNING	가스캐비닛 GC-02	AI run 1/3 완료	폴링 이어서 진행
AI_DONE	스크러버 SCR-01	AI 결과 3종	결과 수정 → 제출 (422 → 200)
PENDING	공정가스 밸브 V-7	AI 결과 3종	안전관리자 승인
REJECTED	펌프 P-208	AI 결과, 거절 이력 3건	거절 사유 확인 → 재제출

상태	설비	같이 들어가는 것	데모 용도
APPROVED	GC-05 (김현장)	AI 결과, 승인 이력	종결 상태
PENDING	SCR-03 (김현장)	AI 결과 3종	안전관리자 거절 (사유 필수 시연)

C. 문서 지도

경로	내용	담당
README.md	실행 방법, 시드 표, Git 컨벤션, R&R	PM
docs/01_planning/	기획서, R&R·일정, 결정 기록	PM
docs/02_usecase/ usecase_spec.md	Actor, Use-Case 13, 다이어그램	UX
docs/03_wireframe/	Figma 와이어프레임, UI 목업, FE 개발 계획서	UX
docs/04_architecture/	구성도, 데이터 흐름(data_flow.md)	PM · API
docs/05_ai_ready/	prompts.md, schemas/, playground_validation.md, validate_schemas.py	장병현
docs/06_erd/	erd.md, fixguide_erd.dbml, schema_postgres.sql, erd.svg	PM(DA)
docs/07_api/README.md	API 명세서 v1.0, Postman Mock URL	정구현
docs/08_presentation/	슬라이드, 데모 대본	PM · 장병현
docs/09_qa/	E2E 검증 보고서, AI-Ready 검토, 회고	DevOps
backend/README.md	스택, 패키지 구조, 환경변수, API 표, Mock 설명	BE

D. 참고 자료

- SKALA Full-Stack Engineering AI 웹 서비스 설계 Mini-project 가이드 (2026-08)
- SK AX 2025-2026 사업·고객 사례 조사 (장병현, 09-02) — AXgenticWire, 대신증권, SBI저축은행, 하이닉스 자율형 펍·GaiA·AMOS, 협력사 1.4조 지원
- SK AX 및 국내 기업 AI 에이전트 운영 Pain Point 조사 (PM, 09-02) — 품질 입증·PoC→운영 격차·책임소재·암묵지·다중 에이전트 비용
- 금융위원회 금융분야 AI 가이드라인 7대 원칙 (2026-06, '보조수단성')
- WRA 화면정의서 v2.0, API 명세서 v1.0, 스캐폴딩·E2E 검증 보고서 (문서 11)